

Experiment Report

(Experiment 4: Random Sampling, Sampling Distributions, and Statistical
Inference with MATLAB)

Class:

Name:

Student ID:

Course Name: 概率和数理统计

Instructor:

Date: 2026.6.25

Experiment 4: Random Sampling, Sampling Distributions, and Statistical Inference with MATLAB

I、 Experiment Objectives

1. Understand the basic concepts of random sampling, sample statistics, sampling distributions, standard error, the Central Limit Theorem, and population mean inference.
2. Use MATLAB simulations to observe the variability of sample statistics, construct the empirical sampling distribution of the sample mean, examine how sample size and population shape affect the spread and shape of the sampling distribution, and use confidence intervals to estimate the population mean.

II、 Experiment Preview

1. Review the following concepts from Chapter 8: population, sample, random sample, sample mean, sample median, sample variance, sample standard deviation, sampling distribution, standard error, normal approximation, the Central Limit Theorem, and confidence interval for the population mean.
2. Learn the lognormal distribution and bimodal distribution used in this experiment. Understand that the parameters of the lognormal distribution describe the normal distribution of the logarithm of the random variable, and that a bimodal distribution has two peaks.
3. Review the following MATLAB functions: `'rand'`, `'randn'`, `'randi'`, `'mean'`, `'exp'`, `'median'`, `'var'`, `'std'`, `'histogram'`, `'plot'`, `'subplot'`, `'xline'`, and `'legend'`.

III、 Experiment Content

This experiment investigates random sampling, sample statistics, sampling distributions, standard error, the Central Limit Theorem, and population mean inference through MATLAB simulation.

A virtual population representing students' weekly mobile phone usage time is generated from a right-skewed lognormal distribution. Random samples are selected from this population to compute sample statistics, including the sample mean, sample median, sample variance, and sample standard deviation. The variability of these statistics across different random samples is observed.

The experiment then uses repeated random sampling to construct the empirical sampling distribution of the sample mean. By changing the sample size and comparing different population shapes, students compare how the spread and shape of the sampling

distribution change, and relate these changes to the standard error and the Central Limit Theorem.

Finally, students use the normal approximation of the sample mean to construct confidence intervals for the population mean. By repeating the confidence interval construction many times, students estimate the simulated coverage probability and compare it with the nominal 95% confidence level.

This experiment consists of four tasks:

1. Population generation and sample statistics
2. Repeated sampling and the sampling distribution of the sample mean
3. Effect of Sample Size, Population Shape, and the Central Limit Theorem
4. Population mean inference using confidence intervals

IV、 Experiment Tasks

(Students should complete all tasks according to the experiment guide and present detailed experimental procedures and results. For each task, students should include relevant source code, tables, figures, and descriptions when applicable.)

1. Task 1 (Population Generation and Sample Statistics)

- 1) Source Code:

```
% Task 1: Population Generation and Sample Statistics
run(fullfile(fileparts(mfilename('fullpath'))), 'common_setup.m'));

% Sample size
n = 30;

% Number of repeated samples
M = 5;

% Plot the histogram of the population
figure;
histogram(population, 'Normalization', 'pdf');
title('Task 1: Histogram of the Population');
xlabel('Weekly Mobile Phone Usage Time (hours)');
ylabel('Density');
grid on;

% Create an array to store sample statistics
% Columns: sample mean, sample median, sample variance, sample
standard deviation
```

```

sample_statistics = zeros(M, 4);

for i = 1:M
    % Hint: generate n random integer indices from 1 to N.
    idx = randi(N, n, 1);

    % Hint: use the indices to sample from population.
    sample = population(idx);

    % Compute sample statistics
    % Hint: compute the sample mean.
    sample_mean = mean(sample);

    % Hint: compute the sample median.
    sample_median = median(sample);

    % Hint: compute the sample variance.
    sample_variance = var(sample);

    % Hint: compute the sample standard deviation.
    sample_std = std(sample);

    % Store the results
    sample_statistics(i, :) = [sample_mean, sample_median,
sample_variance, sample_std];
end

% Display the sample statistics table
Task1_Table = array2table(sample_statistics, ...
    'VariableNames', {'Sample_Mean', 'Sample_Median',
'Sample_Variance', 'Sample_Std'}, ...
    'RowNames', {'Trial_1', 'Trial_2', 'Trial_3', 'Trial_4',
'Trial_5'});

disp('Task 1 Sample Statistics:');
disp(Task1_Table);
disp(['Theoretical population mean mu_X = ', num2str(mu_X)]);

```

2) Description of the code:

- ① Randomly select a sample of size $n = 30$ with replacement from the population dataset.

```

% Sample size
n = 30;

% Number of repeated samples
M = 5;

```

- ② Plot the histogram of the population dataset.

```
% Plot the histogram of the population
figure;
histogram(population, 'Normalization', 'pdf');
title('Task 1: Histogram of the Population');
xlabel('Weekly Mobile Phone Usage Time (hours)');
ylabel('Density');
grid on;
```

- ③ Compute the sample mean, sample median, sample variance, and sample standard deviation.

```
% Create an array to store sample statistics
% Columns: sample mean, sample median, sample variance, sample
standard deviation
sample_statistics = zeros(M, 4);

for i = 1:M
    % Hint: generate n random integer indices from 1 to N.
    idx = randi(N, n, 1);

    % Hint: use the indices to sample from population.
    sample = population(idx);

    % Compute sample statistics
    % Hint: compute the sample mean.
    sample_mean = mean(sample);

    % Hint: compute the sample median.
    sample_median = median(sample);

    % Hint: compute the sample variance.
    sample_variance = var(sample);

    % Hint: compute the sample standard deviation.
    sample_std = std(sample);

    % Store the results
    sample_statistics(i, :) = [sample_mean, sample_median,
sample_variance, sample_std];
end
```

- ④ Display the sample statistics table.

```
% Display the sample statistics table
Task1_Table = array2table(sample_statistics, ...
```

```

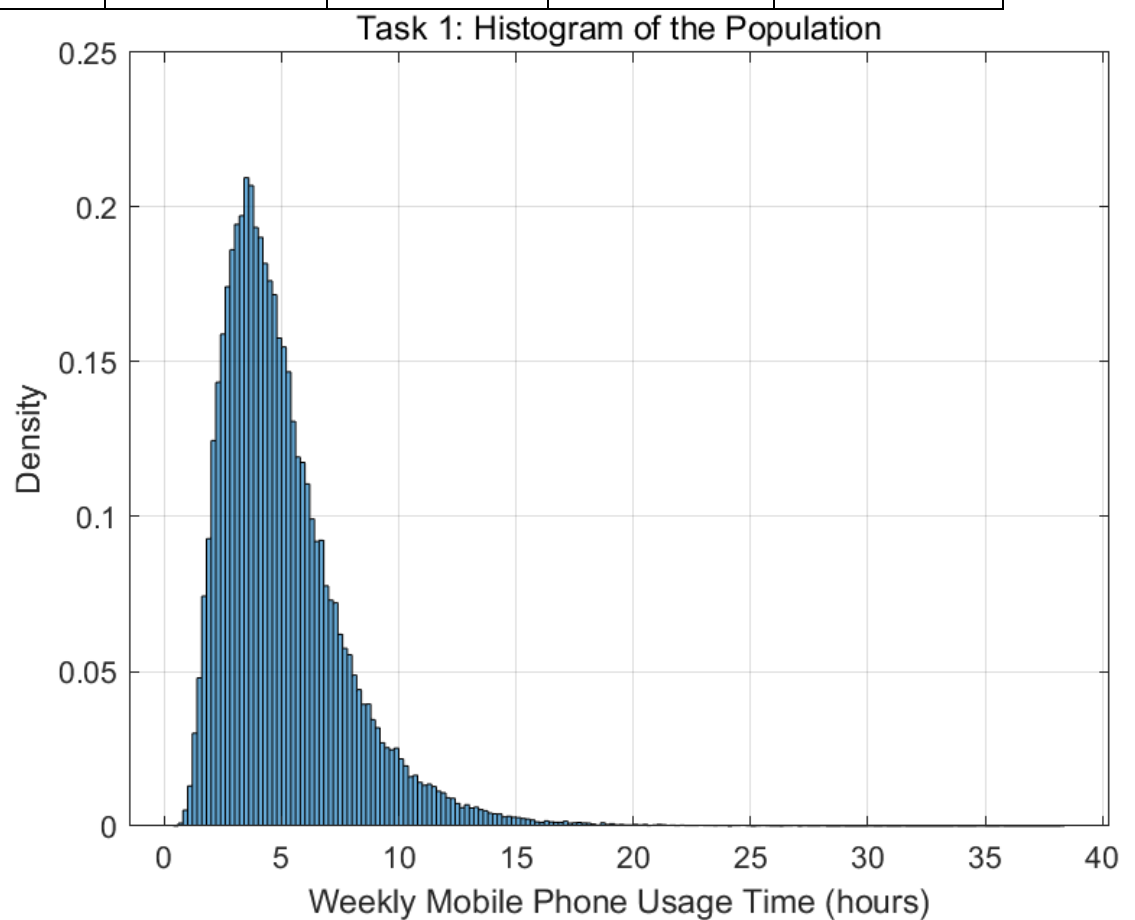
    'VariableNames', {'Sample_Mean', 'Sample_Median',
'Sample_Variance', 'Sample_Std'}, ...
    'RowNames', {'Trial_1', 'Trial_2', 'Trial_3', 'Trial_4',
'Trial_5'}));

disp('Task 1 Sample Statistics:');
disp(Task1_Table);
disp(['Theoretical population mean mu_X = ', num2str(mu_X)]);

```

3) Table and histogram:

Trial	Sample Mean	Sample Median	Sample Variance	Sample Standard Deviation
1	4.3951	4.4491	2.4429	1.563
2	6.2777	5.6622	12.03	3.4684
3	4.8791	4.602	5.2594	2.2933
4	4.7855	4.0338	6.5523	2.5597
5	5.1528	4.0156	9.3372	3.0557



- 4) A short explanation comparing the sample statistics with the population mean:

All five sample means fluctuate randomly around the true population mean μ_X , demonstrating that the sample mean is an **unbiased estimator** of the population mean. Because the lognormal population is right-skewed, every sample mean is noticeably larger than its corresponding sample median. Meanwhile, the sample variances and standard deviations vary considerably across trials, highlighting that dispersion estimates are less stable than central tendency estimates with a moderate sample size of $n = 30$.

2. Task 2 (Sampling Distribution of the Sample Mean)

- 1) Source code:

```
% Task 2: Sampling Distribution of the Sample Mean
run(fullfile(fileparts(mfilename('fullpath')), 'common_setup.m'));

% Sample size
n = 30;

% Number of sampling trials
M = 10000;

% Create an array to store the sample means
sample_means = zeros(M, 1);

for i = 1:M
    % Hint: generate n random integer indices from 1 to N.
    idx = randi(N, n, 1);

    % Hint: use the indices to sample from population.
    sample = population(idx);

    % Hint: compute and store the mean of this sample.
    sample_means(i) = mean(sample);
end

figure;
% Hint: plot the simulated sample means.
histogram(sample_means, 'Normalization', 'pdf');
hold on;
```

```

% Hint: draw a vertical line at the theoretical population mean.
xline(mu_X, 'r-', 'LineWidth', 2);

title('Task 2: Sampling Distribution of the Sample Mean');
xlabel('Sample Mean');
ylabel('Density');
legend('Empirical Sampling Distribution', 'Theoretical Population Mean');
grid on;
hold off;

% Hint: compute the average of all simulated sample means.
mean_of_sample_means = mean(sample_means);

% Display results
disp('Task 2 Results:');
disp(['Theoretical population mean mu_X = ', num2str(mu_X)]);
disp(['Mean of the simulated sample means = ',
num2str(mean_of_sample_means)]);

```

2) Description of the code:

- ① Randomly select a sample of size $n = 30$ with replacement from the population dataset for 10000 times.

```

% Sample size
n = 30;

% Number of sampling trials
M = 10000;

% Create an array to store the sample means
sample_means = zeros(M, 1);

for i = 1:M
    % Hint: generate n random integer indices from 1 to N.
    idx = randi(N, n, 1);

    % Hint: use the indices to sample from population.
    sample = population(idx);

    % Hint: compute and store the mean of this sample.
    sample_means(i) = mean(sample);
end

```

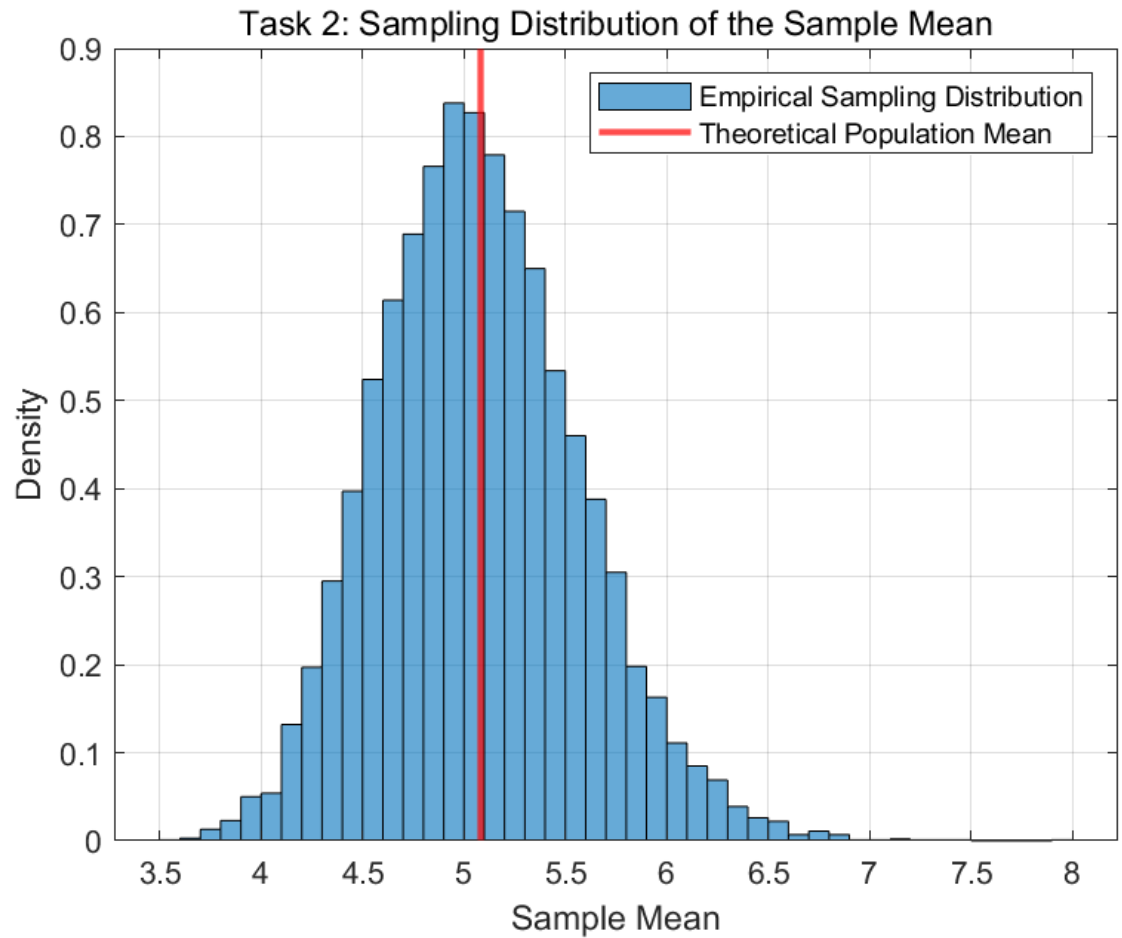

- ② Plot the histogram of the 10000-sample means, and mark the theoretical population mean μ_x on the histogram.

```
figure;  
% Hint: plot the simulated sample means.  
histogram(sample_means, 'Normalization', 'pdf');  
hold on;  
  
% Hint: draw a vertical line at the theoretical population mean.  
xline(mu_X, 'r-', 'LineWidth', 2);  
  
title('Task 2: Sampling Distribution of the Sample Mean');  
xlabel('Sample Mean');  
ylabel('Density');  
legend('Empirical Sampling Distribution', 'Theoretical Population  
Mean');  
grid on;  
hold off;
```

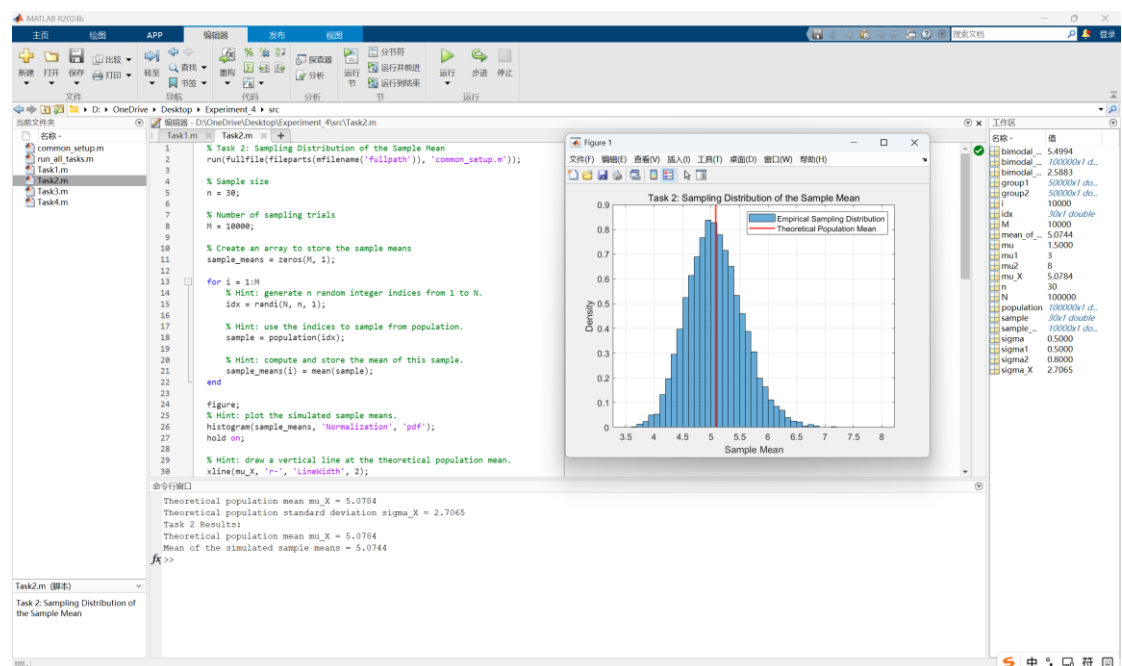
- ③ Display the results.

```
% Display results  
disp('Task 2 Results:');  
disp(['Theoretical population mean mu_X = ', num2str(mu_X)]);  
disp(['Mean of the simulated sample means = ',  
num2str(mean_of_sample_means)]);
```

- 3) Histogram:



4) Screenshot of the terminal output:



- 5) A short explanation of why the sample means are concentrated around the population mean.

The sample means are concentrated around the population mean μ_X because the sample mean is an unbiased estimator of the population mean ($E[\bar{X}] = \mu_X$). Since we repeated the sampling process $M = 10,000$ times, the Law of Large Numbers ensures that the average of all these sample means converges to μ_X , producing a clear bell-shaped peak centered exactly at the theoretical mean.

3. Task 3 (Effect of Sample Size, Population Shape, and the Central Limit Theorem)

- 1) Source code:

```
% Task 3: Effect of Sample Size, Population Shape, and the Central
Limit Theorem
run(fullfile(fileparts(mfilename('fullpath'))), 'common_setup.m'));

% Store the three sample sizes in an array
n_values = [5, 30, 100];

% Number of sampling trials
M = 10000;
```

Part 1: Lognormal Population

```
% Create arrays to store comparison results for the lognormal
population
lognormal_simulated_sd = zeros(length(n_values), 1);
lognormal_theoretical_se = zeros(length(n_values), 1);

figure;

% Use an outer loop to go through the three sample sizes
for j = 1:length(n_values)
    n = n_values(j);

    % Create an array to store sample means
    sample_means = zeros(M, 1);

    % Repeat the sampling process M times
    for i = 1:M
```

```

        idx = randi(N, n, 1);
        sample = population(idx);

        % Hint: compute and store the mean of this sample.
        sample_means(i) = mean(sample);
    end

    % Hint: compute the standard deviation of the simulated sample
    means.
    lognormal_simulated_sd(j) = std(sample_means);

    % Hint: use the formula  $\sigma_X / \sqrt{n}$ .
    lognormal_theoretical_se(j) = sigma_X / sqrt(n);

    % Plot the histogram of sample means
    subplot(3, 1, j);
    histogram(sample_means, 'Normalization', 'pdf');
    hold on;

    % Generate x-axis values for the normal density curve
    x = linspace(mu_X - 4 * lognormal_theoretical_se(j), ...
        mu_X + 4 * lognormal_theoretical_se(j), 200);

    % Hint: use the normal density formula with mean mu_X and standard
    error lognormal_theoretical_se(j).
    y = 1./((lognormal_theoretical_se(j) * sqrt(2*pi)) .* ...
        exp(-0.5 * ((x - mu_X) ./ lognormal_theoretical_se(j)).^ 2));

    % Plot the normal density curve
    plot(x, y, 'r-', 'LineWidth', 2);

    title(['Lognormal Population, n = ', num2str(n)]);
    xlabel('Sample Mean');
    ylabel('Density');
    legend('Empirical Distribution', 'Normal Density Curve');
    grid on;
    hold off;
end

```

Part 2: Bimodal Population

```

% Hint: compute the mean of the bimodal population.
bimodal_mu_X = mean(bimodal_population);
% Hint: compute the standard deviation of the bimodal population.
bimodal_sigma_X = std(bimodal_population);

% Create arrays to store comparison results for the bimodal population

```

```

bimodal_simulated_sd = zeros(length(n_values), 1);
bimodal_theoretical_se = zeros(length(n_values), 1);

figure;

% Use the same outer loop structure for the bimodal population
for j = 1:length(n_values)
    n = n_values(j);

    % Create an array to store sample means
    sample_means = zeros(M, 1);

    % Repeat the sampling process M times
    for i = 1:M
        idx = randi(N, n, 1);
        sample = bimodal_population(idx);

        % Hint: compute and store the mean of this sample.
        sample_means(i) = mean(sample);
    end

    % Hint: compute the standard deviation of the simulated sample
    means.
    bimodal_simulated_sd(j) = std(sample_means);

    % Hint: use the formula bimodal_sigma_X / sqrt(n).
    bimodal_theoretical_se(j) = bimodal_sigma_X / sqrt(n);

    % Plot the histogram of sample means
    subplot(3, 1, j);
    histogram(sample_means, 'Normalization', 'pdf');
    hold on;

    % Generate x-axis values for the normal density curve
    x = linspace(bimodal_mu_X - 4*bimodal_theoretical_se(j), ...
        bimodal_mu_X + 4*bimodal_theoretical_se(j), 200);

    % Hint: use the normal density formula with mean bimodal_mu_X and
    standard error bimodal_theoretical_se(j).
    y = 1./(bimodal_theoretical_se(j) * sqrt(2*pi)) .* ...
        exp(-0.5 * ((x - bimodal_mu_X) ./ bimodal_theoretical_se(j)).^
2);

    % Plot the normal density curve
    plot(x, y, 'r-', 'LineWidth', 2);

    title(['Bimodal Population, n = ', num2str(n)]);
    xlabel('Sample Mean');
    ylabel('Density');

```

```

legend('Empirical Distribution', 'Normal Density Curve');
grid on;
hold off;
end

```

Display the comparison table

```

Population_Type = {'Lognormal'; 'Lognormal'; 'Lognormal'; ...
    'Bimodal'; 'Bimodal'; 'Bimodal'};

Sample_Size_n = [n_values'; n_values'];

Simulated_SD = [lognormal_simulated_sd; bimodal_simulated_sd];

Theoretical_SE = [lognormal_theoretical_se; bimodal_theoretical_se];

Task3_Table = table(Population_Type, Sample_Size_n, Simulated_SD,
Theoretical_SE);

disp('Task 3 Comparison Table:');
disp(Task3_Table);

```

2) Description of the code:

- ① For each sample size n , perform 10000 sampling trials using the population dataset.

```

% Store the three sample sizes in an array
n_values = [5, 30, 100];

% Number of sampling trials
M = 10000;

```

- ② In each trial, randomly select n values with replacement, compute their mean, and store it as one sample mean.

```

% Create arrays to store comparison results for the lognormal
population
lognormal_simulated_sd = zeros(length(n_values), 1);
lognormal_theoretical_se = zeros(length(n_values), 1);

figure;

% Use an outer loop to go through the three sample sizes

```

```

for j = 1:length(n_values)
    n = n_values(j);

    % Create an array to store sample means
    sample_means = zeros(M, 1);

    % Repeat the sampling process M times
    for i = 1:M
        idx = randi(N, n, 1);
        sample = population(idx);

        % Hint: compute and store the mean of this sample.
        sample_means(i) = mean(sample);
    end
end

```

- ③ Compute the simulated standard deviation of the 10000 sample means and compare it with the theoretical standard error $SE = \frac{\sigma_X}{\sqrt{n}}$.

```

% Hint: compute the standard deviation of the simulated sample means.
lognormal_simulated_sd(j) = std(sample_means);

% Hint: use the formula sigma_X / sqrt(n).
lognormal_theoretical_se(j) = sigma_X / sqrt(n);

```

- ④ For each n, plot the histogram of the 10000 sample means.

```

% Plot the histogram of sample means
subplot(3, 1, j);
histogram(sample_means, 'Normalization', 'pdf');
hold on;

% Generate x-axis values for the normal density curve
x = linspace(mu_X - 4 * lognormal_theoretical_se(j), ...
    mu_X + 4 * lognormal_theoretical_se(j), 200);

% Hint: use the normal density formula with mean mu_X and standard error lognormal_theoretical_se(j).
y = 1./((lognormal_theoretical_se(j) * sqrt(2*pi)) .* ...
    exp(-0.5 * ((x - mu_X) ./ lognormal_theoretical_se(j)).^ 2));

% Plot the normal density curve
plot(x, y, 'r-', 'LineWidth', 2);

title(['Lognormal Population, n = ', num2str(n)]);
xlabel('Sample Mean');
ylabel('Density');

```

```

legend('Empirical Distribution', 'Normal Density Curve');
grid on;
hold off;
end

```

- ⑤ Repeat for the bimodal population.

```

% Hint: compute the mean of the bimodal population.
bimodal_mu_X = mean(bimodal_population);
% Hint: compute the standard deviation of the bimodal population.
bimodal_sigma_X = std(bimodal_population);

% Create arrays to store comparison results for the bimodal population
bimodal_simulated_sd = zeros(length(n_values), 1);
bimodal_theoretical_se = zeros(length(n_values), 1);

figure;

% Use the same outer loop structure for the bimodal population
for j = 1:length(n_values)
    n = n_values(j);

    % Create an array to store sample means
    sample_means = zeros(M, 1);

    % Repeat the sampling process M times
    for i = 1:M
        idx = randi(N, n, 1);
        sample = bimodal_population(idx);

        % Hint: compute and store the mean of this sample.
        sample_means(i) = mean(sample);
    end

    % Hint: compute the standard deviation of the simulated sample means.
    bimodal_simulated_sd(j) = std(sample_means);

    % Hint: use the formula bimodal_sigma_X / sqrt(n).
    bimodal_theoretical_se(j) = bimodal_sigma_X / sqrt(n);

    % Plot the histogram of sample means
    subplot(3, 1, j);
    histogram(sample_means, 'Normalization', 'pdf');
    hold on;

    % Generate x-axis values for the normal density curve
    x = linspace(bimodal_mu_X - 4*bimodal_theoretical_se(j), ...

```



```

        bimodal_mu_X + 4*bimodal_theoretical_se(j), 200);

    % Hint: use the normal density formula with mean bimodal_mu_X and
    standard error bimodal_theoretical_se(j).
    y = 1./((bimodal_theoretical_se(j) * sqrt(2*pi)) .* ...
        exp(-0.5 * ((x - bimodal_mu_X) ./ bimodal_theoretical_se(j)) .^
2);

    % Plot the normal density curve
    plot(x, y, 'r-', 'LineWidth', 2);

    title(['Bimodal Population, n = ', num2str(n)]);
    xlabel('Sample Mean');
    ylabel('Density');
    legend('Empirical Distribution', 'Normal Density Curve');
    grid on;
    hold off;
end

```

⑥ Display the comparison table.

```

Population_Type = {'Lognormal'; 'Lognormal'; 'Lognormal'; ...
    'Bimodal'; 'Bimodal'; 'Bimodal'};

Sample_Size_n = [n_values'; n_values'];

Simulated_SD = [lognormal_simulated_sd; bimodal_simulated_sd];

Theoretical_SE = [lognormal_theoretical_se; bimodal_theoretical_se];

Task3_Table = table(Population_Type, Sample_Size_n, Simulated_SD,
Theoretical_SE);

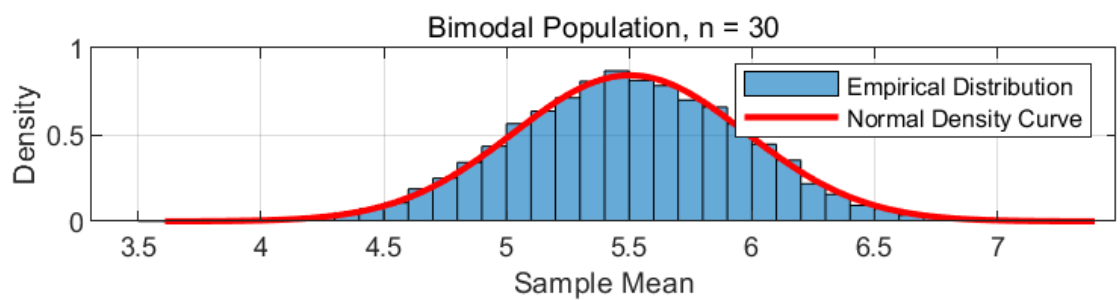
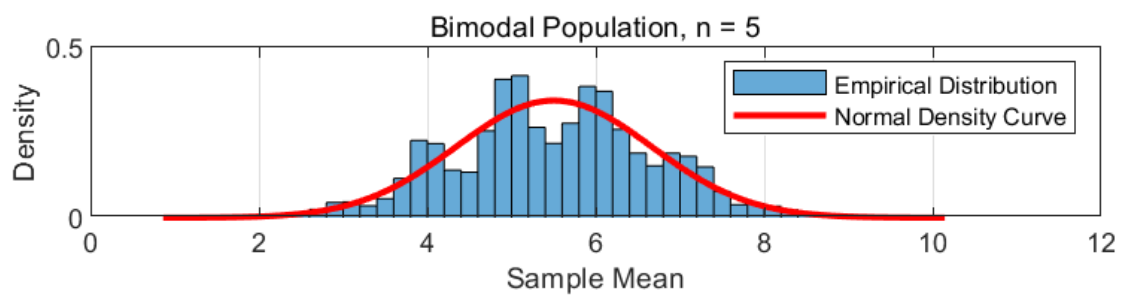
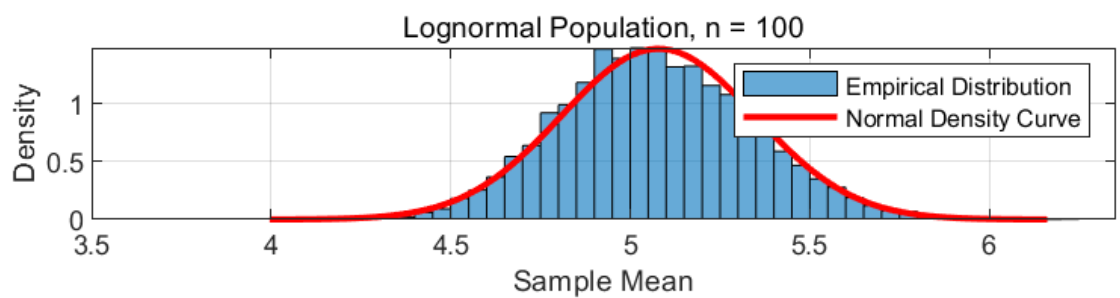
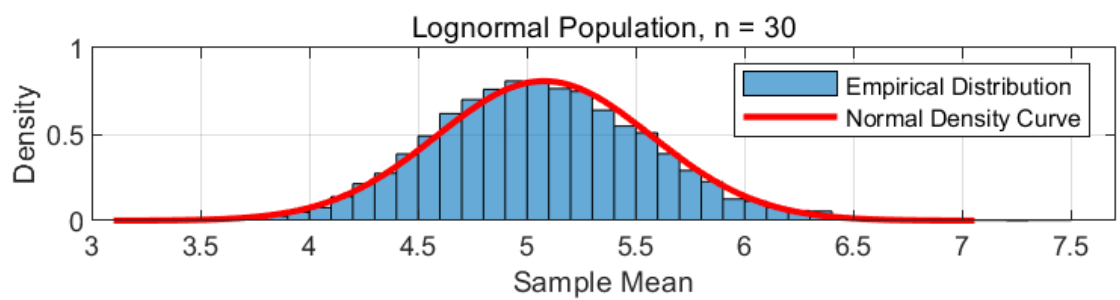
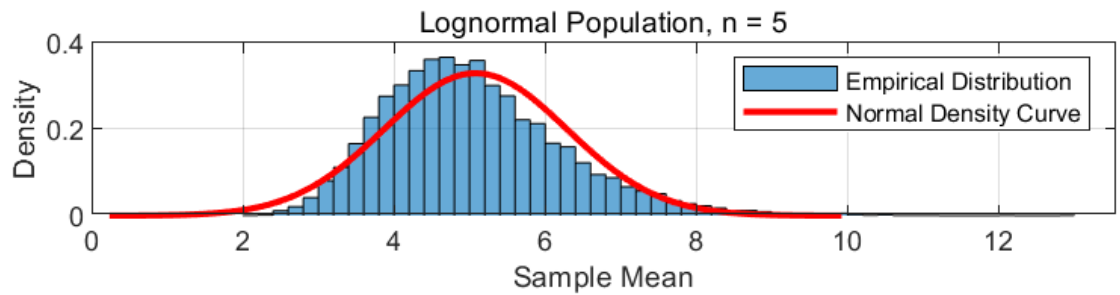
disp('Task 3 Comparison Table:');
disp(Task3_Table);

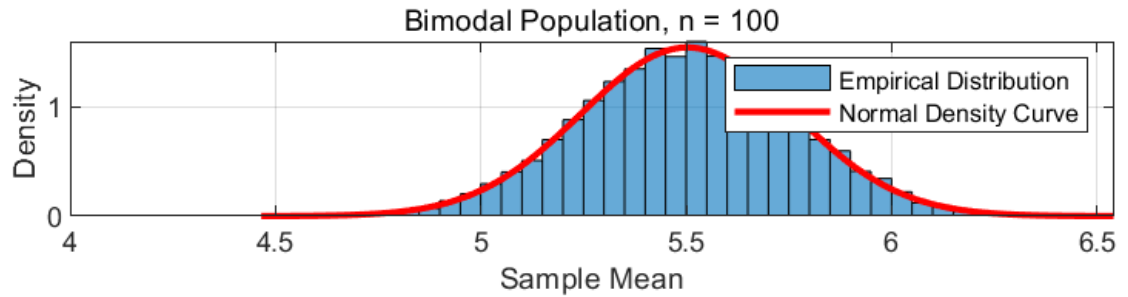
```

3) Table and histogram:

Population Type	Sample Size n	Simulated SD of 10000 Sample Means	Theoretical $SE = \sigma_X / \sqrt{n}$
Lognormal	5	1.2092	1.2104
	30	0.49937	0.49414
	100	0.26878	0.27065

Bimodal	5	1.1631	1.16
	30	0.46767	0.47355
	100	0.26067	0.25937





- 4) Short explanation of how the sample size affects both the spread and the shape of the sampling distribution of the sample mean.

Increasing the sample size n significantly reduces the spread of the sampling distribution, as the standard error $\left(\frac{\sigma}{\sqrt{n}}\right)$ becomes smaller—this makes the distribution tighter and more concentrated around the true mean. More importantly, the shape transforms dramatically: with small n (like 5), the distribution retains the skewness or bimodality of the original population, but as n grows (to 30 and 100), it gradually evolves into a symmetric, bell-shaped normal curve, perfectly illustrating the Central Limit Theorem (CLT) in action.

4. Task 4 (Population Mean Inference Using Confidence Intervals)

- 1) Source code:

```
% Task 4: Population Mean Inference Using Confidence Intervals
run(fullfile(fileparts(mfilename('fullpath'))), 'common_setup.m'));

% Sample size
n = 30;

% Number of trials
M = 100000;

% Count the number of intervals containing mu_X
coverage_count = 0;

% Hint: compute the 95% margin of error: 1.96 * sigma_X / sqrt(n).
margin_error = 1.96 * sigma_X / sqrt(n);

% Generate one example confidence interval
```

```

idx = randi(N, n, 1);
sample = population(idx);

% Hint: compute the sample mean.
x_bar = mean(sample);

% Hint: construct the lower bound of the confidence interval.
lower_bound = x_bar - margin_error;

% Hint: construct the upper bound of the confidence interval.
upper_bound = x_bar + margin_error;

% Store the example confidence interval
example_sample_mean = x_bar;
example_lower_bound = lower_bound;
example_upper_bound = upper_bound;

% Repeat the process M times
for i = 1:M
    % Random sampling with replacement
    idx = randi(N, n, 1);
    sample = population(idx);

    % Compute the sample mean
    x_bar = mean(sample);

    % Construct the confidence interval
    lower_bound = x_bar - margin_error;
    upper_bound = x_bar + margin_error;

    % Hint: check whether mu_X is inside the confidence interval.
    if (mu_X >= lower_bound) && (mu_X <= upper_bound)
        coverage_count = coverage_count + 1;
    end
end

% Hint: compute the proportion of intervals that contain mu_X.
coverage_probability = coverage_count / M;

% Display results
disp('Task 4 Example 95% Confidence Interval:');
disp(['Sample mean = ', num2str(example_sample_mean)]);
disp(['Lower bound = ', num2str(example_lower_bound)]);
disp(['Upper bound = ', num2str(example_upper_bound)]);
disp(['Theoretical population mean mu_X = ', num2str(mu_X)]);

disp('Task 4 Simulated Coverage Probability:');
disp(['Number of trials M = ', num2str(M)]);

```

```

disp(['Number of intervals containing mu_X = ',
num2str(coverage_count)]);
disp(['Simulated coverage probability = ',
num2str(coverage_probability)]);

```

2) Description of the code:

- ① In each trial, randomly select a sample of size $n = 30$ with replacement from the population dataset.

```

% Sample size
n = 30;

```

- ② Compute the sample mean and construct the 95% confidence interval using $\bar{X} \pm 1.96 \frac{\sigma_X}{\sqrt{n}}$.

```

% Hint: compute the 95% margin of error: 1.96 * sigma_X / sqrt(n).
margin_error = 1.96 * sigma_X / sqrt(n);

% Generate one example confidence interval
idx = randi(N, n, 1);
sample = population(idx);

% Hint: compute the sample mean.
x_bar = mean(sample);

% Hint: construct the lower bound of the confidence interval.
lower_bound = x_bar - margin_error;

% Hint: construct the upper bound of the confidence interval.
upper_bound = x_bar + margin_error;

% Store the example confidence interval
example_sample_mean = x_bar;
example_lower_bound = lower_bound;
example_upper_bound = upper_bound;

% Repeat the process M times
for i = 1:M
    % Random sampling with replacement
    idx = randi(N, n, 1);
    sample = population(idx);

    % Compute the sample mean

```

```
x_bar = mean(sample);

% Construct the confidence interval
lower_bound = x_bar - margin_error;
upper_bound = x_bar + margin_error;
```

- ③ Record whether the confidence interval contains the theoretical population mean μ_X .

```
% Hint: check whether mu_X is inside the confidence interval.
if (mu_X >= lower_bound) && (mu_X <= upper_bound)
    coverage_count = coverage_count + 1;
end
end
```

- ④ After 100000 trials, compute the proportion of intervals that contain μ_X .

```
% Hint: compute the proportion of intervals that contain mu_X.
coverage_probability = coverage_count / M;
```

- ⑤ Display results.

```
% Display results
disp('Task 4 Example 95% Confidence Interval:');
disp(['Sample mean = ', num2str(example_sample_mean)]);
disp(['Lower bound = ', num2str(example_lower_bound)]);
disp(['Upper bound = ', num2str(example_upper_bound)]);
disp(['Theoretical population mean mu_X = ', num2str(mu_X)]);

disp('Task 4 Simulated Coverage Probability:');
disp(['Number of trials M = ', num2str(M)]);
disp(['Number of intervals containing mu_X = ',
num2str(coverage_count)]);
disp(['Simulated coverage probability = ',
num2str(coverage_probability)]);
```

- 3) Table:

Sample Size n	Number of Trials M	Number of Intervals Containing μ_X	Simulated Coverage Probability
30	100000	94963	0.94963

- 4) One 95% confidence interval from a single random sample, with its sample mean, lower bound, and upper bound.

```

1 % Task 4: Population Mean Inference Using Confidence Intervals
2 run(fullfile(fileparts(mfilename('fullpath'))), 'common_setup.m');
3
4 % Sample size
5 n = 30;
6
7 % Number of trials
8 M = 100000;
9
10 % Count the number of intervals containing mu_X
11 coverage_count = 0;
12
13 % Hint: compute the 95% margin of error: 1.96 * sigma_X / sqrt(n).
14 margin_error = 1.96 * sigma_X / sqrt(n);
15
16 % Generate one example confidence interval
17 idx = randi(N, n, 1);
18 sample = population(idx);
19
20 % Hint: compute the sample mean.
21 x_bar = mean(sample);
22
23 % Hint: construct the lower bound of the confidence interval.
24 lower_bound = x_bar - margin_error;
25
26 % Hint: construct the upper bound of the confidence interval.
27 upper_bound = x_bar + margin_error;
28
29 % Store the example confidence interval
30 example_sample_mean = x_bar;

```

命令窗口

```

Theoretical population mean mu_X = 5.0784
Theoretical population standard deviation sigma_X = 2.7065
Task 4 Example 95% Confidence Interval:
Sample mean = 5.0651
Lower bound = 4.0966
Upper bound = 6.0336
Theoretical population mean mu_X = 5.0784
Task 4 Simulated Coverage Probability:
Number of trials M = 100000
Number of intervals containing mu_X = 95180
Simulated coverage probability = 0.9518

```

- 5) A short explanation of why the simulated coverage probability is close to 95%.

The simulated coverage probability is close to 95% because the **Central Limit Theorem (CLT)** ensures that the sample mean $\bar{X}$ approximately follows a normal distribution centered at μ_X with standard error $\left(\frac{\sigma}{\sqrt{n}}\right)$. Consequently, the interval $\bar{X} \pm 1.96 \cdot \left(\frac{\sigma}{\sqrt{n}}\right)$ exactly captures the middle 95% of this normal curve. With a large number of trials ($M = 100,000$), the **Law of Large Numbers** ensures that the observed frequency of captured intervals converges to this true theoretical probability of 0.95, leaving only negligible random fluctuation.

V、 Extended Questions

In this experiment, the samples are generated by random sampling with replacement from a simulated population. In real surveys, however, samples may be biased. For example, if a survey on weekly mobile phone usage is conducted only among students in a computer lab, the sample may not represent the whole student population.

Question:

If the sampling process is biased, will increasing the sample size still guarantee an accurate estimate of the population mean? Explain your answer using the ideas of random sampling, sampling distribution, and the Central Limit Theorem.

No. Increasing sample size **cannot** compensate for sampling bias.

If sampling is biased, the expected value of the sample mean becomes μ_{biased} , not the true population mean μ_{true} . Thus, as n grows, $\bar{X}$ converges precisely to the **wrong target**. The CLT only ensures normality of $\bar{X}$ around that biased centre; it does not remove systematic error. Increasing n reduces random error (variance) but cannot fix systematic bias. In short, without random sampling, more data only gives more precise incorrect estimates.

VI、 Summary and Experiment Reflections

This experiment bridged probability theory with computational simulation by using MATLAB to generate random data and numerically verify the core concepts of expectation, variance, covariance, and risk. Across five tasks, I confirmed that manual formulas for sample mean and variance (with denominator $(N - 1)$) perfectly match MATLAB's built-in functions, which reinforced the importance of using the correct degrees of freedom in estimation.

The histogram-based Riemann sum approximation in Task 2 showed that both sample size and bin count affect numerical accuracy: larger samples reduce random fluctuation, while finer bins minimise discretisation error, bringing the Riemann sum closer to the true expected value. Task 3 demonstrated that while expectation is linear ($E[aX + bY] = aE[X] + bE[Y]$), variance of a linear combination must include the covariance term $2ab \text{Cov}(X, Y)$; omitting it severely underestimates the true variability when variables are correlated.

The last two tasks revealed that the mean alone is a poor measure of risk. The St. Petersburg game produced a final sample mean around 26, but its enormous variance and rare huge payoffs made the cumulative average unstable and highly dependent on extreme events—even though theory promises infinite expectation. Similarly, the 1D random walk kept the group average near zero, yet individual paths wandered far from the origin, and the variance grew linearly with time, proving that uncertainty expands indefinitely even in a “fair” game.

Overall, this experiment transformed abstract formulas into visible, interpretable

results. I learned that statistical inference requires looking beyond the average: variance and covariance quantify the spread and dependence that are essential for understanding risk and making informed decisions under uncertainty. The simulations also sharpened my MATLAB skills in handling arrays, extracting histogram properties, and using cumulative statistics to observe convergence behaviour in random processes.